

Provenance of the Agent-Execution Trace Dataset

Inspect AI `.eval` traces collected for the study of LLM-agent success predictability

César Ferri Ramírez

Compiled 28 September 2026

Purpose of this document

This report documents the origin, collection method, format and current storage location of the agent-execution trace dataset built over June–July 2026 (and consolidated during a research stay at the Leverhulme Centre for the Future of Intelligence, University of Cambridge). It is intended as a provenance record: it explains *where each trace comes from, how it was produced or imported, and what it contains*, so that the dataset can be reused, audited or extended without having to reconstruct this history from scratch.

All traces are stored in the **Inspect AI** `.eval` format (<https://inspect.aisi.org.uk/>), a structured, sample-level evaluation log that preserves, for every task attempted by an agent: the full sequence of raw LLM requests and responses (system/user/assistant/tool messages, exactly as sent to and received from the model), tool calls and their outputs, timestamps per call, and the final pass/fail (or scored) outcome assigned by the benchmark’s own grader.

1 Dataset overview

The dataset is assembled from three independent sources, summarised in Table 1.

Source	What was run	Benchmarks	Models	Size
A. RCB / API models	OpenClaw agent vs. commercially hosted LLMs (Azure)	5	8	113 MB, 50 <code>.eval</code>
B. RCB / open-weight models	Same harness vs. locally served open-weight LLMs (SLURM/vLLM)	1 (RCB)	9	~700 MB
C. HAL import	Decrypted third-party traces from Princeton’s HAL leaderboard	9	16 families	4.9 GB, 334 <code>.eval</code>

Table 1: The three sources that make up the dataset.

Sources A and B share the same benchmark harness (**Real-Claw-Bench**, RCB) and the same agent runtime (**OpenClaw**); they differ only in how the LLM being evaluated is served. Source C is external and independently produced by the HAL project; it was imported and reformatted, not generated by us.

2 Source A — RCB via commercial APIs (Azure)

2.1 What it is

Real-Claw-Bench (RCB) is an agentic coding benchmark in which the **OpenClaw** agent is given a coding task inside an isolated workspace and must complete it autonomously (editing files, running commands, invoking tools) until a per-task verifier judges the result. Five benchmark suites were run under this harness:

Benchmark	Tasks	Notes
Terminal-Bench-Core	42	general terminal/agentive tasks
SWE-bench-Verified	50	real-world GitHub issue resolution
QuixBugs	40	program-repair (Python only; Java tasks unbuildable)
Real-Claw-Bench (native)	281	the harness’s own task suite
Aider-Polyglot	225	multi-language coding tasks

2.2 Models

Eight LLMs were evaluated, served through two Azure endpoints reached via a local reverse proxy:

- **Azure OpenAI:** GPT-4o, GPT-4.1, GPT-5.4, o1 (o1 and GPT-5.4 are reasoning models);
- **Azure AI Foundry:** Llama-4-Maverick, Mistral-Large-3, DeepSeek-v3.2, Grok-4.3.

2.3 How it was produced

The benchmark originally depended on a local GPU/SLURM cluster running vLLM, a fragile setup (the vLLM asynchronous engine would occasionally die on an iteration timeout and bring an entire run down). In June 2026 it was re-engineered to talk to Azure-hosted models instead, through a purpose-built local proxy (`azure_openai_proxy.py`) that:

- translates the OpenAI-style requests OpenClaw emits into the Azure convention (deployment name embedded in the URL, `api-key` header, `api-version` query parameter);
- for the o-series and GPT-5 family, renames `max_tokens` to `max_completion_tokens` and strips sampling parameters (temperature, top-p, penalties) that Azure rejects for reasoning models;
- retries transient errors (HTTP 429/5xx) with exponential backoff, honouring the `Retry-After` header, so provider-side rate limiting does not corrupt results;
- never writes the API key to disk (read from the environment only).

Every run of every (benchmark, model) pair was logged with Inspect AI and exported to a browsable `.eval` file (`inspect view`).

2.4 Known gaps

- **Aider-Polyglot:** complete (225/225) for GPT-4o, GPT-4.1, DeepSeek, Llama-4, o1. GPT-5.4 has 224/225 (one C++ task’s tests never parse in the harness). Mistral-Large-3 (128/225) and Grok-4.3 (3/225) were blocked by an expired Azure Foundry key and were not completed.
- **USACO** and **AppWorld** were attempted but dropped: USACO’s verbose task preamble caused LLM request timeouts before any solution was written; AppWorld’s post-test synchronisation hung for ~40 minutes per task inside the harness (an unrelated bug in the test-wait logic), making it impractically slow.

2.5 Storage

`falco:~/sw/real-claw-bench/inspect_logs/(tbench/,azure/,azure50/,azure_v2/,azure_v3/ subdirectories, by run batch); mirrored locally under sw/real-claw-bench-582B/inspect_logs/`

3 Source B — RCB via open-weight models (HPC cluster)

The same RCB/OpenClaw harness was also run against nine open-weight models served locally with vLLM on the DSIC SLURM cluster (`vrhpcadm1`), using GPU nodes reserved through SLURM batch jobs: Qwen2.5-7B/14B/32B, Qwen2.5-Coder-7B/14B/32B, Mistral-Nemo-12B, Mistral-Small-22B and DeepSeek-Coder-V2-Lite. This is the configuration Source A eventually replaced for the API-served models, but it remains the only way to evaluate open-weight models that are not available through a commercial

API. Raw per-task JSONL logs live under `cluster:~/rcb/runs/hpc*` and `traces/hpc*`; the corresponding exported `.eval` files are mirrored locally under `sw/real-claw-bench-582B/inspect_logs/` (one file per model, e.g. `qwen2-5-32b.eval`).

4 Source C — Imported HAL traces

4.1 What it is

The **Holistic Agent Leaderboard** (HAL, Princeton, hal.cs.princeton.edu) independently runs a wide range of agent scaffolds against a wide range of benchmarks and publishes the resulting execution traces as an encrypted dataset on HuggingFace: `agent-evals/hal_traces` (380 zip archives, ~113 GB total). This source was added to broaden the dataset beyond what we could run ourselves, both in benchmark coverage and in agent/model diversity.

4.2 Import pipeline

Each HuggingFace zip contains one JSON object `{salt, encrypted_data}`; the payload is Fernet ciphertext derived via PBKDF2-HMAC-SHA256 (480,000 iterations) from a password published by the HAL project. A pipeline (`import_hal_traces.py`) streams each zip from HuggingFace, decrypts it, converts it to Inspect `.eval`, and deletes the temporary zip, one archive at a time to bound memory use. “Full” traces carry the complete list of raw LLM completion calls (request + response + token usage); “slim” traces carry labels only and are skipped. Per-task pass/fail labels follow four different schema variants across HAL benchmarks (a dict of successful/failed task IDs, a per-task success-rate dict, a positional list of per-task scores, or a positional list of correctness floats); the importer normalises all four into a single label format.

4.3 Coverage

334 valid `.eval` files were produced (0 hard errors), from 46 archives that were skipped as unrecoverable (39 empty at the source, 5 labels-only, 1 exceeding a memory guard, 1 without usable labels).

Benchmark	Traces imported	Domain
tau-bench	47	customer-service / tool-use agents
CoreBench	60 (6 skipped)	computational research reproduction
SWE-bench-mini	45	GitHub issue resolution
SciCode	38	scientific coding
ColBench	36	backend/frontend collaborative coding
GAIA	35 (2 skipped)	general assistant reasoning
AssistantBench	33 (1 skipped)	realistic web-browsing tasks
USACO	15	competitive-programming
BrowserUse	0 (19 skipped)	web-browsing agent (mostly empty traces)

Across the imported traces, model identity was recovered for **16 distinct model families** (spanning multiple providers: OpenAI, Anthropic, Google, DeepSeek, and others, as used by the original HAL runs).

4.4 Storage

`falco:~/hal/inspect_logs/` (4.9 GB) and `~/hal/inspect_logs_colbench/` (a later corrective re-import of ColBench, merged into the main directory); mirrored locally under `sw/real-claw-bench-582B/`

5 Consolidated analysis dataset

For the predictability study, all three sources above were flattened into a single task-level table, `apriori_rows.jsonl` (on `falco:~/pred_difficulty/`), containing **46,486 task-level rows** across the HAL benchmarks,

spanning **16 model families** with an overall pass rate of 31.7%. (Sources A and B are tracked separately and were used mainly for infrastructure validation and as a second, independently produced corpus; the headline predictability numbers are computed on the HAL-derived table.)

6 Access summary

Source	Remote location (<code>falco.dsic.upv.es</code> , unless noted)	Local mirror
A. RCB / API models	<code>~/sw/real-claw-bench/inspect_logs/</code>	<code>sw/real-claw-bench-582B/inspect_logs/</code> (tbench/, azure*/)
B. RCB / open-weight	<code>cluster:~/rcb/runs/hpc*</code> and <code>traces/hpc*</code>	<code>sw/real-claw-bench-582B/inspect_logs/ (*.eval)</code>
C. HAL import	<code>~/hal/inspect_logs/</code>	<code>sw/real-claw-bench-582B/inspect_logs/hal/</code>
Consolidated table	<code>~/pred_difficulty/apriori_rows.jsonl</code>	(not yet mirrored)

Note on licensing/reuse: Source C is derived from a third-party dataset (HAL, Princeton); any reuse or redistribution of those traces should respect the licence terms published alongside `agent-evals/hal_traces` on HuggingFace. Sources A and B were produced entirely in-house and carry no external licence restrictions.